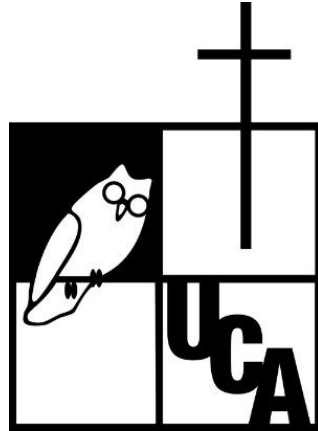


Documento técnico - Sistema de Gestión Clínica Veterinaria



Alonzo Gómez, Astrid Adamaris (00091525)

Castillo Oliva, Susan Abigail (00221125)

Espinal Echegoyen, Daniel Alexander (00100725)

Guadrón Vásquez, Francisco Javier (00077225)

Mejía Méndez, Adriana Paola (00098225)

Facultad De Ingeniería Y Arquitectura

Universidad Centroamericana “José Simeon Cañas”

01-2026: Bases De Datos

Ing. James Humberstone

19 de junio de 2026

Antiguo Cuscatlán, El Salvador, C.A



Contenido

1.	Descripción del escenario	3
2.	Reglas del negocio.....	4
3.	Diagrama entidad relación.....	5
4.	Esquema relacional normalizado (3FN) (Modelo entidad relación)	6
5.	Diccionario de datos	7
6.	Documentación	14
6.1.	Consultas frecuentes	14
6.1.1.	Medicamentos prescritos con mayor frecuencia	14
6.1.2.	Dinero ganado por cada especialidad médica.....	15
6.1.3.	propietarios que más dinero han generado mediante citas y facturas.....	16
6.1.4.	Mascotas con cumpleaños en el mes actual y contacto del propietario.....	17
6.1.5.	Veterinario con mayor número de citas en el trimestre	18
6.1.6.	Motivos de consulta frecuentes y su asignación de tratamiento.....	19
6.2.	Funciones	20
6.2.1.	fn_get_edad	20
6.2.2.	fn_get_historial_medico	21
6.3.	Procedimientos almacenados	23
6.3.1.	Registrar veterinario con especialidad.....	23
6.3.2.	Agregar medicamento o actualizar stock existente	25
6.4.	Disparadores	27
6.4.1.	Control de entidades débiles (Secuenciadores)	27
6.4.1.1.	Automatización para DETALLES_FACTURAS	27
6.4.1.2.	Automatización para DETALLES_TRATAMIENTOS.....	28
6.4.2.	Trigger para sumar el total de la factura automáticamente.....	28
6.4.3.	Reducción de stock de medicamentos	29
7.	Anexos	31

1. Descripción del escenario

La Clínica Veterinaria "**Fast paws**" es un centro médico especializado en la atención integral de animales de compañía. La clínica ofrece servicios de consulta general, diagnóstico clínico, tratamientos farmacológicos y procedimientos médicos especializados, atendidos por un equipo de veterinarios con distintas especialidades, como medicina general, cirugía, odontología, entre otras.

El sistema gestiona el ciclo completo de atención de una mascota: desde el registro del propietario y su animal, pasando por la programación y atención de citas médicas, hasta la emisión de facturas por los servicios prestados.

Cuando un propietario llega a la clínica, se registra su información personal junto con la de su mascota, indicando la especie y raza del animal. A partir de ahí, se pueden agendar citas médicas que son asignadas a un veterinario según su especialidad. Durante cada cita, el veterinario emite uno o más diagnósticos, a partir de los cuales puede recomendar procedimientos médicos y prescribir tratamientos que incluyen uno o varios medicamentos del inventario de la clínica. Al finalizar la consulta, el sistema genera automáticamente una factura que acumula el costo de todos los servicios e insumos utilizados, y descuenta del inventario los medicamentos dispensados.

El sistema también permite a la clínica llevar un seguimiento del historial clínico de cada mascota, identificar los medicamentos más recetados para optimizar las compras, analizar los ingresos por especialidad médica, y contactar a los propietarios de mascotas que cumplen años en el mes vigente para fidelizar a los clientes.



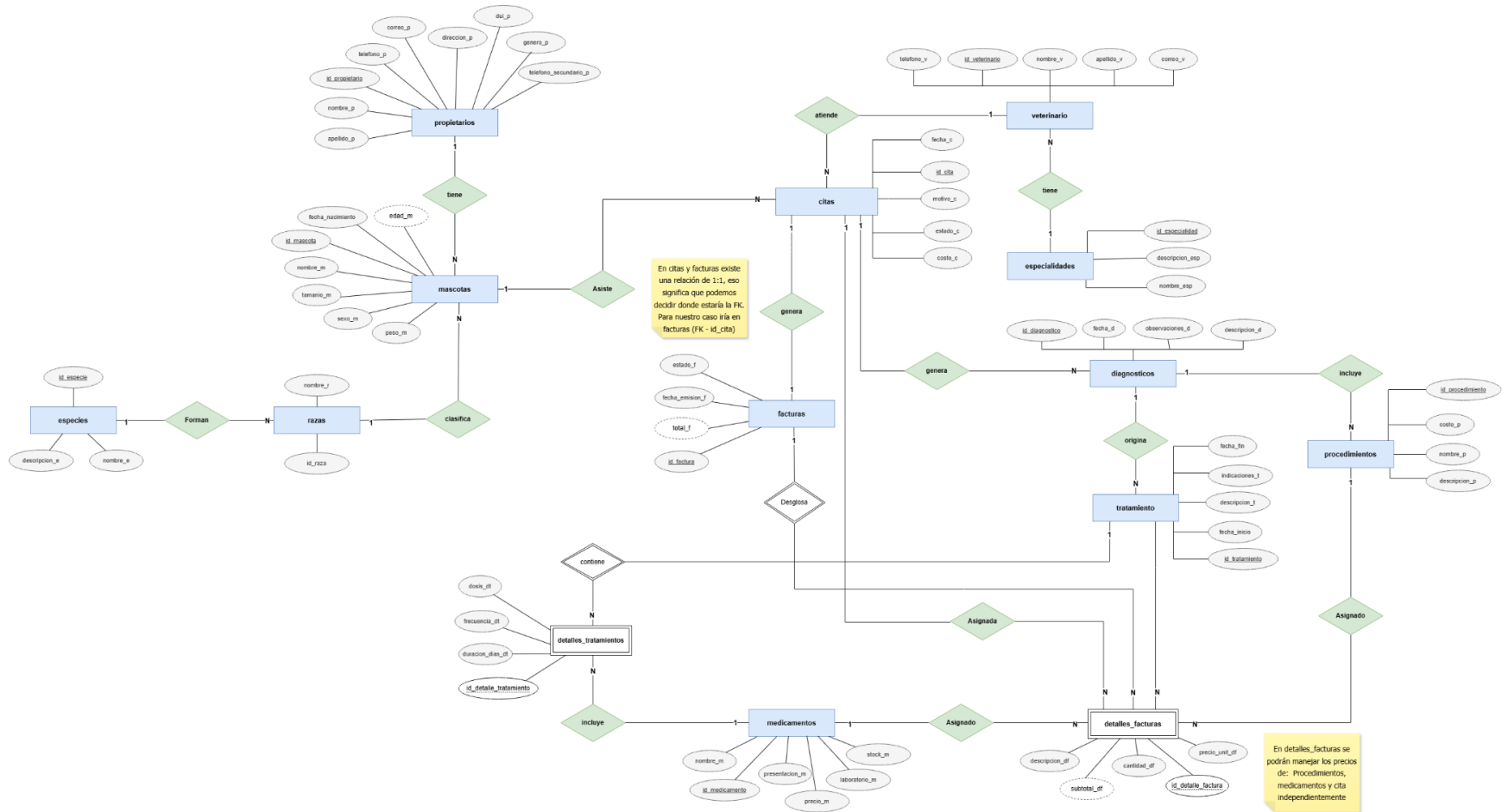
2. Reglas del negocio

El presente apartado establece las reglas de negocio que definen la estructura de datos del sistema de gestión veterinaria, garantizando la consistencia e integridad de la información almacenada.

- Cada propietario puede registrar una o más mascotas dentro del sistema.
- Cada especie es conformada por múltiples tipos de razas, mientras que cada mascota pertenece a una única raza.
- Una mascota puede asistir a una o más citas médicas a lo largo de su historial clínico.
- Cada cita médica es atendida por un veterinario, quien posee una especialidad profesional específica.
- Una cita genera múltiples diagnósticos asociados a la consulta.
- Un diagnóstico puede incluir varios procedimientos médicos recomendados durante la consulta.
- Cada tratamiento es generado a partir de un diagnóstico y puede estar conformado por uno o múltiples medicamentos prescritos.
- Cada cita genera una factura en base a los servicios prestados.



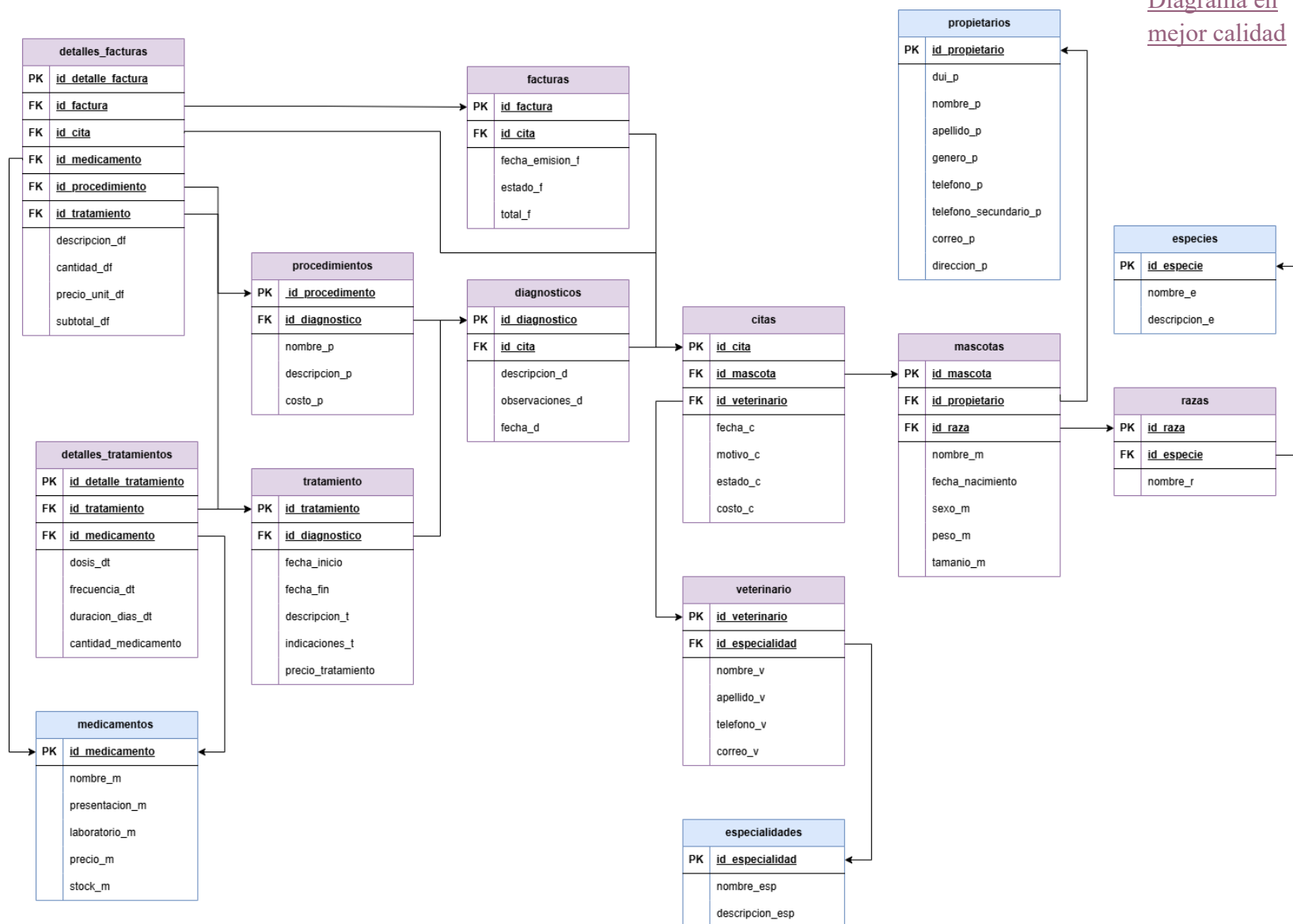
3. Diagrama entidad relación



[Link para ver el diagrama con mejor calidad](#)



4. Esquema relacional normalizado (3FN) (Modelo entidad relación)



5. Diccionario de datos

El presente diccionario de base de datos describe la estructura lógica del sistema de gestión para una clínica veterinaria, implementado en PostgreSQL. Su propósito es documentar de forma clara y precisa cada una de las tablas que conforman el modelo relacional.

La base de datos se organiza en 14 tablas interrelacionadas que abarcan los principales procesos operativos de la clínica: el registro de especies y razas de animales, la gestión de propietarios y mascotas, la administración del personal veterinario y sus especialidades, el control de citas médicas, el registro de diagnósticos, procedimientos y tratamientos, el inventario de medicamentos, y la facturación de servicios. Cada tabla ha sido diseñada aplicando principios de normalización y buenas prácticas relacionales, con el uso de claves primarias de identidad autogenerada, claves foráneas con políticas de actualización y eliminación controladas, y restricciones de validación que garantizan la consistencia de los datos en todo momento.

1) TABLA ESPECIES

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_especie PK	Identificador único de la especie.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
nombre_e	Nombre de la especie animal.	VARCHAR	100	NOT NULL · UNIQUE
descripcion_e	Descripción detallada de la especie.	TEXT	Ilimitado	NOT NULL

2) TABLA RAZAS

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_raza PK	Identificador único de la raza.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
id_especie FK	Especie a la que pertenece la raza.	BIGINT	—	NOT NULL · Ref. especies(id_especie) · ON UPDATE CASCADE · ON DELETE RESTRICT
nombre_r	Nombre de la raza.	VARCHAR	100	NOT NULL · UNIQUE compuesto con id_especie



3) TABLA PROPIETARIOS

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_propietario PK	Identificador único del propietario.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
dui_p	Número de identificación personal (DUI).	VARCHAR	20	NOT NULL · UNIQUE · LENGTH(TRIM()) > 0
nombre_p	Nombre del propietario.	VARCHAR	100	NOT NULL
apellido_p	Apellido del propietario.	VARCHAR	100	NOT NULL
genero_p	Género del propietario.	VARCHAR	10	Opcional (NULL permitido) · IN ('M', 'F', 'Otro')
telefono_p	Teléfono principal del propietario.	VARCHAR	20	NOT NULL · UNIQUE · telefono_p <> telefono_secundario_p
telefono_secundario_p	Teléfono secundario del propietario.	VARCHAR	20	NOT NULL · telefono_p <> telefono_secundario_p
correo_p	Correo electrónico del propietario.	VARCHAR	150	Opcional (NULL permitido) · UNIQUE · LIKE '%@%.%'
direccion_p	Dirección de residencia.	TEXT	Ilimitado	Opcional (NULL permitido)

4) TABLA MASCOTAS

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_mascota PK	Identificador único de la mascota.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
id_propietario FK	Propietario al que pertenece la mascota.	BIGINT	—	NOT NULL · Ref. propietarios(id_propietario) · ON UPDATE CASCADE · ON DELETE RESTRICT
id_raza FK	Raza de la mascota.	BIGINT	—	NOT NULL · Ref. razas(id_raza) · ON UPDATE CASCADE · ON DELETE RESTRICT
nombre_m	Nombre de la mascota.	VARCHAR	100	NOT NULL · LENGTH(TRIM()) > 0
fecha_nacimiento	Fecha de nacimiento de la mascota.	DATE	—	Opcional (NULL permitido) · fecha_nacimiento <= CURRENT_DATE
sexo_m	Sexo de la mascota.	VARCHAR	10	Opcional (NULL permitido) · IN ('Macho', 'Hembra')



peso_m	Peso corporal en kilogramos.	DECIMAL	6,2	Opcional (NULL permitido) · peso_m > 0
tamano_m	Tamaño corporal de la mascota.	VARCHAR	50	Opcional (NULL permitido)

5) TABLA ESPECIALIDADES

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_especialidad PK	Identificador único de la especialidad.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
nombre_esp	Nombre de la especialidad.	VARCHAR	150	NOT NULL · UNIQUE
descripcion_esp	Descripción de la especialidad.	TEXT	Ilimitado	Opcional (NULL permitido)

6) TABLA VETERINARIO

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_veterinario PK	Identificador único del veterinario.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
id_especialidad FK	Especialidad del veterinario.	BIGINT	—	NOT NULL · Ref. especialidades(id_especialidad) · ON UPDATE CASCADE · ON DELETE RESTRICT
nombre_v	Nombre del veterinario.	VARCHAR	100	NOT NULL · LENGTH(TRIM()) > 0
apellido_v	Apellido del veterinario.	VARCHAR	100	NOT NULL · LENGTH(TRIM()) > 0
telefono_v	Teléfono del veterinario.	VARCHAR	20	Opcional (NULL permitido) · UNIQUE
correo_v	Correo electrónico del veterinario.	VARCHAR	150	Opcional (NULL permitido) · UNIQUE · LIKE '%@%.%'



7) TABLA CITAS

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_cita PK	Identificador único de la cita.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
id_mascota FK	Mascota asociada a la cita.	BIGINT	—	NOT NULL · Ref. mascotas(id_mascota) · ON UPDATE CASCADE · ON DELETE RESTRICT
id_veterinario FK	Veterinario que atiende la cita.	BIGINT	—	NOT NULL · Ref. veterinario(id_veterinario) · ON UPDATE CASCADE · ON DELETE RESTRICT
fecha_c	Fecha y hora de la cita.	TIMESTAMP	—	NOT NULL · Si estado='Programada': fecha_c >= NOW() - INTERVAL '1 hour'
motivo_c	Motivo de la consulta.	TEXT	Ilimitado	Opcional (NULL permitido)
estado_c	Estado actual de la cita.	VARCHAR	20	NOT NULL · DEFAULT 'Programada' · IN ('Programada', 'En curso', 'Completada', 'Cancelada')
costo_c	Costo de la cita.	DECIMAL	10,2	Opcional (NULL permitido) · costo_c >= 0

8) TABLA DIAGNOSTICOS

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_diagnostico PK	Identificador único del diagnóstico.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
id_cita FK	Cita médica vinculada al diagnóstico.	BIGINT	—	NOT NULL · UNIQUE · Ref. citas(id_cita) · ON UPDATE CASCADE · ON DELETE RESTRICT
descripcion_d	Descripción detallada del diagnóstico clínico.	TEXT	Ilimitado	NOT NULL · LENGTH(TRIM())) > 0
observaciones_d	Observaciones adicionales del veterinario.	TEXT	Ilimitado	Opcional (NULL permitido)
fecha_d	Fecha en que se emite el diagnóstico.	DATE	—	Opcional (NULL permitido) · fecha_d <= CURRENT_DATE



9) TABLA PROCEDIMIENTOS

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_procedimiento PK	Identificador único del procedimiento.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
id_diagnostico FK	Diagnóstico al que pertenece el procedimiento.	BIGINT	—	NOT NULL · Ref. diagnosticos(id_diagnostico) · ON UPDATE CASCADE · ON DELETE RESTRICT
nombre_p	Nombre del procedimiento realizado.	VARCHAR	100	Opcional (NULL permitido) · LENGTH(Trim()) > 0 si no es NULL
descripcion_p	Descripción técnica del procedimiento.	TEXT	Ilimitado	Opcional (NULL permitido)
costo_p	Costo monetario del procedimiento.	DECIMAL	10,2	Opcional (NULL permitido) · costo_p >= 0

10) TABLA TRATAMIENTO

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_tratamiento PK	Identificador único del tratamiento.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
id_diagnostico FK	Diagnóstico que origina el tratamiento.	BIGINT	—	NOT NULL · Ref. diagnosticos(id_diagnostico) · ON UPDATE CASCADE · ON DELETE RESTRICT
fecha_inicio	Fecha de inicio del tratamiento.	DATE	—	Opcional (NULL permitido) · fecha_inicio >= '2000-01-01' · fecha_inicio <= fecha_fin si ambas existen
fecha_fin	Fecha de finalización del tratamiento.	DATE	—	Opcional · Requiere fecha_inicio si se registra · fecha_fin >= fecha_inicio
descripcion_t	Descripción general del tratamiento.	TEXT	Ilimitado	Opcional (NULL permitido)
indicaciones_t	Instrucciones para el propietario.	TEXT	Ilimitado	Opcional (NULL permitido)



11) TABLA MEDICAMENTOS

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_medamento PK	Identificador único del medicamento.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
nombre_m	Nombre comercial o genérico del medicamento.	VARCHAR	150	NOT NULL · LENGTH(TRIM()) > 0 · UNIQUE compuesto con presentacion_m
presentacion_m	Forma farmacéutica (tableta, inyectable, etc.).	VARCHAR	100	Opcional (NULL permitido) · UNIQUE compuesto con nombre_m
laboratorio_m	Laboratorio fabricante.	VARCHAR	150	Opcional (NULL permitido)
precio_m	Precio unitario del medicamento.	DECIMAL	10,2	NOT NULL · DEFAULT 0 · precio_m >= 0
stock_m	Cantidad disponible en inventario.	INT	—	NOT NULL · DEFAULT 0 · stock_m >= 0

12) TABLA DETALLES_TRATAMIENTOS

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_detalle_tratamiento PK	Identificador del detalle (parte de PK compuesta). Controlado por trigger.	BIGINT	—	NOT NULL · PK compuesta con id_tratamiento
id_tratamiento PK FK	Tratamiento al que pertenece el detalle.	BIGINT	—	NOT NULL · PK compuesta · Ref. tratamiento(id_tratamiento) · ON UPDATE CASCADE · ON DELETE CASCADE
id_medamento FK	Medicamento incluido en el detalle.	BIGINT	—	NOT NULL · UNIQUE con id_tratamiento · Ref. medicamentos(id_medamento) · ON UPDATE CASCADE · ON DELETE RESTRICT
dosis_dt	Dosis indicada del medicamento.	VARCHAR	100	Opcional (NULL permitido)
frecuencia_dt	Frecuencia de administración (ej. cada 8 horas).	VARCHAR	100	Opcional (NULL permitido)
duracion_dias_dt	Duración del tratamiento con ese medicamento.	INT	—	Opcional (NULL permitido) · 1 <= duracion_dias_dt <= 365

13) TABLA FACTURAS

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_factura PK	Identificador único de la factura.	BIGINT	—	GENERATED ALWAYS AS IDENTITY · PRIMARY KEY
id_cita FK	Cita médica asociada a la factura.	BIGINT	—	Opcional (NULL permitido) · UNIQUE · Ref. citas(id_cita) · ON UPDATE CASCADE · ON DELETE RESTRICT
fecha_emision_f	Fecha de emisión de la factura.	DATE	—	NOT NULL · fecha_emision_f <= CURRENT_DATE
estado_f	Estado actual de la factura.	VARCHAR	50	DEFAULT 'Pendiente' · IN ('Pendiente', 'Pagada', 'Anulada', 'Cancelada') · Si estado='Pagada': total_f > 0
total_f	Monto total de la factura.	DECIMAL	10,2	DEFAULT 0 · total_f >= 0

14) TABLA DETALLES_FACTURAS

CAMPO	DESCRIPCIÓN	TIPO DE DATO	TAMAÑO	VALIDACIÓN Y RESTRICCIÓN
id_detalle_factura PK	Identificador del detalle (parte de PK compuesta). Controlado por trigger.	BIGINT	—	NOT NULL · PK compuesta con id_factura
id_factura PK FK	Factura a la que pertenece el detalle.	BIGINT	—	NOT NULL · PK compuesta · Ref. facturas(id_factura) · ON UPDATE CASCADE · ON DELETE CASCADE
id_cita FK	Cita vinculada al ítem (si aplica).	BIGINT	—	Opcional (NULL permitido) · Ref. citas(id_cita) · ON UPDATE CASCADE · ON DELETE SET NULL
id_medimento FK	Medicamento facturado (si aplica).	BIGINT	—	Opcional (NULL permitido) · Ref. medicamentos(id_medimento) · ON UPDATE CASCADE · ON DELETE SET NULL
id_tratamiento FK	Tratamiento facturado (si aplica).	BIGINT	—	Opcional (NULL permitido) · Ref. tratamientos(id_tratamiento) · ON UPDATE CASCADE · ON DELETE SET NULL
id_procedimiento FK	Procedimiento facturado (si aplica).	BIGINT	—	Opcional (NULL permitido) · Ref. procedimientos(id_procedimiento) ·



				ON UPDATE CASCADE · ON DELETE SET NULL
descripcion_df	Descripción del ítem facturado.	TEXT	Ilimitado	NOT NULL · LENGTH(TRIM()) > 0
cantidad_df	Cantidad de unidades del ítem.	BIGINT	—	NOT NULL · DEFAULT 1 · cantidad_df > 0
precio_unit_df	Precio unitario del ítem.	DECIMAL	10,2	NOT NULL · DEFAULT 0 · precio_unit_df >= 0
subtotal_df	Subtotal calculado automáticamente.	DECIMAL	10,2	GENERATED ALWAYS AS (cantidad_df * precio_unit_df) STORED · No editable directamente
— (restricción)	Solo uno de los 4 FKs opcionales puede ser NOT NULL a la vez.	—	—	CHECK: exactamente un FK de referencia poblado por fila (chk_un_solo_item)

6. Documentación

6.1. Consultas frecuentes

6.1.1. Medicamentos prescritos con mayor frecuencia

Objetivo: Identificar el patrón de consumo farmacéutico dentro de la clínica para optimizar las órdenes de compra a proveedores.

Lógica: Junta las tablas de recetas (detalles_tratamientos) y medicinas (medicamentos), cuenta cuántas veces aparece cada una usando COUNT() y ordena la lista de mayor a menor.

```
SELECT
    med.id_medicamento,
    med.nombre_m AS medicamento,
    med.presentacion_m AS presentacion,
    COUNT(dt.id_medicamento) AS veces_recetado
FROM detalles_tratamientos dt
JOIN medicamentos med ON dt.id_medicamento = med.id_medicamento
GROUP BY med.id_medicamento, med.nombre_m, med.presentacion_m
ORDER BY veces_recetado DESC;
```

6.1.2. Dinero ganado por cada especialidad médica

Objetivo: Ver cuánto dinero le ha sumado a la clínica cada área de especialidad (Cirugía, medicina general, odontología, etc.).

Lógica:

1. En la base de datos, una fila de la factura cobra una consulta sola o cobra un procedimiento, pero nunca las dos cosas en la misma línea. El uso del LEFT JOIN en vez del INNER JOIN es debido a que se borraría de la consulta cualquier fila que tenga un campo vacío, en cambio con LEFT JOIN se protegen las filas y hace que no se pierda ni un solo dólar del reporte, aunque haya campos vacíos.
2. Como el dinero puede venir por dos caminos (una cita normal o una cirugía que salió de un diagnóstico), el COALESCE revisa cuál de las dos opciones tiene el ID del doctor encargado, agarra al veterinario correcto y le suma el dinero a su especialidad.

```
SELECT
    esp.nombre_esp AS especialidad,
    SUM(df.subtotal_df) AS ingresos_totales
FROM detalles_facturas df
LEFT JOIN citas c ON df.id_cita = c.id_cita
LEFT JOIN procedimientos proc ON df.id_procedimiento = proc.id_procedimiento
LEFT JOIN diagnosticos d ON proc.id_diagnostico = d.id_diagnostico
LEFT JOIN citas c_proc ON d.id_cita = c_proc.id_cita
JOIN veterinario v ON v.id_veterinario = COALESCE(c.id_veterinario, c_proc.id_veterinario)
JOIN especialidades esp ON v.id_especialidad = esp.id_especialidad
GROUP BY esp.nombre_esp
ORDER BY ingresos_totales DESC;
```



6.1.3. propietarios que más dinero han generado mediante citas y facturas

Objetivo: Identificar los propietarios que mayor ingreso han generado para la clínica veterinaria, considerando únicamente las facturas con estado Pagada, con el fin de reconocer a los clientes más valiosos económicamente.

Lógica:

La consulta parte de la tabla propietarios y navega la cadena de relaciones hasta llegar a facturas: un propietario tiene mascotas, cada mascota puede tener citas, y cada cita puede tener asociada una factura.

Se aplica el filtro WHERE f.estado_f = 'Pagada' para excluir facturas pendientes, anuladas o canceladas, garantizando que solo se contabilicen ingresos efectivamente cobrados.

Luego se agrupa por propietario y se calculan dos métricas: el número total de facturas pagadas (COUNT) y la suma de los montos de esas facturas (SUM). Finalmente, el ORDER BY ingresos_generados DESC presenta los resultados de mayor a menor ingreso, colocando al cliente más rentable en la primera posición.

```
SELECT
    p.id_propietario,
    INITCAP(CONCAT(p.nombre_p, ' ', p.apellido_p)) AS propietario,
    COUNT(f.id_factura) AS total_facturas,
    SUM(f.total_f) AS ingresos_generados
FROM propietarios p
JOIN mascotas m
    ON p.id_propietario = m.id_propietario
JOIN citas c
    ON m.id_mascota = c.id_mascota
JOIN facturas f
    ON c.id_cita = f.id_cita
WHERE f.estado_f = 'Pagada'
GROUP BY p.id_propietario, p.nombre_p, p.apellido_p
ORDER BY ingresos_generados DESC;
```



6.1.4. Mascotas con cumpleaños en el mes actual y contacto del propietario

Objetivo: Obtener la lista de mascotas que cumplen años en el mes actual junto con los datos de contacto de su propietario, permitiendo a la clínica comunicarse con los dueños para felicitarlos o enviarles promociones.

Lógica:

1. Se usa `DATE_PART('month', fecha_nacimiento)` comparado con `DATE_PART('month', CURRENT_DATE)` para filtrar únicamente las mascotas que cumplen años en el mes actual, sin importar el año de nacimiento.
2. `DATE_PART('year', AGE(fecha_nacimiento))` calcula la edad actual de la mascota en años cumplidos.
3. Se hace JOIN con propietarios para traer el nombre completo, teléfono y correo del dueño, convirtiendo la consulta en una herramienta de contacto directo.
4. Se ordena por día del mes con `DATE_PART('day', fecha_nacimiento)` para presentar los cumpleaños en orden cronológico dentro del mes.

```
SELECT
    m.nombre_m,
    TO_CHAR(m.fecha_nacimiento, 'DD/MM') AS cumpleaños,
    DATE_PART('year', AGE(m.fecha_nacimiento)) AS edad_actual,
    p.nombre_p || ' ' || p.apellido_p AS propietario,
    p.telefono_p,
    p.correo_p
FROM mascotas m
JOIN propietarios p ON m.id_propietario = p.id_propietario
WHERE DATE_PART('month', m.fecha_nacimiento) = DATE_PART('month',
CURRENT_DATE)
```



6.1.5. Veterinario con mayor número de citas en el trimestre

Objetivo: Identificar qué veterinario atendió la mayor cantidad de citas en cada trimestre del año, permitiendo evaluar la carga de trabajo del personal y reconocer a los profesionales más activos por período.

Lógica:

1. Se unen las tablas citas, veterinario y especialidades mediante JOIN para obtener el nombre del profesional que atendió cada cita. Se excluyen con WHERE las citas en estado Cancelada y Programada, ya que no representan trabajo real ejecutado.
2. La función DATE_PART extrae el año y trimestre de cada cita (1 = Ene-Mar, 2 = Abr-Jun, 3 = Jul-Sep, 4 = Oct-Dic), permitiendo al GROUP BY agrupar las citas de cada veterinario dentro de su período correspondiente. Sobre cada grupo, COUNT totaliza las citas atendidas.
3. El ORDER BY ordena los resultados del trimestre más reciente al más antiguo, colocando siempre al veterinario con más citas en la parte superior de cada período.

```
SELECT
  CONCAT(v.nombre_v, ' ', v.apellido_v) AS veterinario,
  DATE_PART('year', c.fecha_c)      AS anio,
  DATE_PART('quarter', c.fecha_c)   AS trimestre,
  COUNT(c.id_cita)                  AS total_citas
FROM citas c
JOIN veterinario v    ON c.id_veterinario = v.id_veterinario
JOIN especialidades esp ON v.id_especialidad = esp.id_especialidad
WHERE c.estado_c IN ('Completada', 'En curso')
GROUP BY
  v.nombre_v,
  v.apellido_v,
  esp.nombre_esp,
  DATE_PART('year', c.fecha_c),
  DATE_PART('quarter', c.fecha_c)
ORDER BY
  anio DESC,
  trimestre DESC,
  total_citas DESC;
```



6.1.6. Motivos de consulta frecuentes y su asignación de tratamiento.

Objetivo: Analizar patrones patológicos y tendencias epidemiológicas para optimizar el abastecimiento de medicamentos y facilitar la detección temprana de posibles brotes de enfermedades.

Lógica:

1. Se unen las tablas citas, diagnósticos, tratamientos, detalles_tratamientos y medicamentos mediante un LEFT JOIN. Se seleccionó este tipo de unión para garantizar que se muestren las citas completadas, incluso cuando no exista información asociada en alguna de las tablas relacionadas.
2. Se aplica STRING_AGG para concatenar todos los tratamientos asociados a un mismo motivo de consulta. demás se agregó la función COALESCE para reemplazar con 'Sin tratamiento asignado' en los campos dónde los datos fueran NULL para facilitar la interpretación de los resultados. La misma lógica se aplica para la columna medicamento_recetado.
3. El WHERE se utiliza para filtrar las citas cuyo estado es 'Completada'. Debido a que en esta fase están definidos todos los tratamientos y medicamentos recetados, lo que permite realizar un análisis de patrones epidemiológicos más preciso.
4. La función COUNT(DISTINCT c.id_cita) calcula el número de casos únicos para cada motivo de consulta, evitando la multiplicidad de datos, relacionados con tratamientos y medicamentos.
5. De la misma forma el ORDER BY ordena los resultados mediante los casos epidemiológicos registrados de manera descendente.



```

SELECT
    c.motivo_c AS motivo_consulta,
    COUNT(DISTINCT c.id_cita) AS casos,
    COALESCE(STRING_AGG(DISTINCT t.descripcion_t, ' | '), 'Sin tratamiento asignado')
        AS      tratamientos_asignados,
    COALESCE(STRING_AGG(DISTINCT m.nombre_m, ' | '), 'Sin medicamento asignado')
        AS      medicamento_recetado
FROM citas c
LEFT JOIN diagnosticos d ON d.id_cita = c.id_cita
LEFT JOIN tratamiento t ON t.id_diagnostico = d.id_diagnostico
LEFT JOIN detalles_tratamientos dt ON dt.id_tratamiento = t.id_tratamiento
LEFT JOIN medicamentos m ON m.id_medimento = dt.id_medimento
WHERE c.estado_c = 'Completada'
GROUP BY c.motivo_c
ORDER BY casos DESC;

```

6.2. Funciones

6.2.1. fn_get_edad

Objetivo:

Calcular la edad actual de una mascota en años completos a partir de su fecha de nacimiento, retornando 0 si el valor es NULL.

Lógica:

Emplea `EXTRACT (YEAR FROM age(current_date, f_fecha_nacimiento))` para obtener la diferencia en años entre la fecha actual y la fecha de nacimiento. El resultado se envuelve en `COALESCE` para garantizar que nunca se retorne NULL: si la fecha es nula o el cálculo falla, la función devuelve 0.



```

CREATE OR REPLACE FUNCTION fn_get_edad(f_fecha_nacimiento date)

    RETURNS int

    LANGUAGE plpgsql

AS $$

DECLARE

    c_edad bigint;

BEGIN

    c_edad = EXTRACT(YEAR FROM age(current_date, f_fecha_nacimiento));

    RETURN coalesce(c_edad, 0);

END;

$$;

```

6.2.2. fn_get_historial_medico

Objetivo

Retornar el historial clínico completo de una mascota específica, consolidando en un único conjunto de resultados sus citas, el veterinario que la atendió, los diagnósticos emitidos, los tratamientos prescritos y los medicamentos recetados con su dosis.

Lógica

Ejecuta un RETURN QUERY que encadena 7 tablas mediante JOINs (mascotas → citas → veterinario → diagnosticos → tratamiento → detalles_tratamientos → medicamentos). Reutiliza fn_get_edad para calcular la edad del paciente. Aplica string_agg para concatenar todos los medicamentos de un mismo tratamiento en una sola cadena con formato nombre (dosis), agrupando por todos los campos no agregados mediante GROUP BY.



```

CREATE OR REPLACE FUNCTION fn_get_historial_medico(id int)
RETURNS TABLE(
    nombre_ma          text,
    edad               int,
    peso               numeric(6,2),
    fecha_cita         timestamp,
    veterinario         text,
    diagnostico         text,
    tratamiento         text,
    medicamentos_recetados text
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN query
    SELECT
        m.nombre_m::text,
        fn_get_edad(m.fecha_nacimiento)::int,
        m.peso_m,
        c.fecha_c,
        v.nombre_v::text,
        d.descripcion_d::text,
        t.descripcion_t::text,
        string_agg(
            me.nombre_m || ' (' || dt.dosis_dt || ')',
            ','
        )::text AS medicamentos_recetados
    FROM mascotas m
    JOIN citas      c ON c.id_mascota    = m.id_mascota
    JOIN veterinario v ON v.id_veterinario = c.id_veterinario
    JOIN diagnosticos d ON d.id_cita      = c.id_cita
    JOIN tratamiento t ON t.id_diagnostico = d.id_diagnostico
    JOIN detalles_tratamientos dt ON dt.id_tratamiento = t.id_tratamiento
    JOIN medicamentos me ON me.id_medicamento = dt.id_medicamento
    WHERE m.id_mascota = id
    GROUP BY
        m.nombre_m,
        m.fecha_nacimiento,
        m.peso_m,
        c.fecha_c,
        d.descripcion_d,
        t.descripcion_t,
        v.nombre_v;
END;
$$;

```



6.3. Procedimientos almacenados

6.3.1. Registrar veterinario con especialidad

Objetivo: Registrar un nuevo veterinario en el sistema junto con su especialidad médica, evitando duplicados tanto en especialidades como en datos de contacto del profesional.

Parámetros de entrada:

Parámetro	Tipo	Descripción
p_nombre_esp	VARCHAR (150)	Nombre de la especialidad médica
p_descripcion_esp	TEXT	Descripción de la especialidad
p_nombre_v	VARCHAR (100)	Nombre del veterinario
p_apellido_v	VARCHAR (100)	Apellido del veterinario
p_telefono_v	VARCHAR (20)	Teléfono de contacto del veterinario
p_correo_v	VARCHAR (150)	Correo electrónico del veterinario

Lógica:

1. Valida que el nombre de la especialidad no esté vacío; si lo está, lanza una excepción controlada.
2. Valida que el nombre y apellido del veterinario no estén vacíos.
3. Verifica que el correo electrónico no esté ya registrado en la tabla '**veterinarios**'; si existe, lanza una excepción.
4. Verifica que el teléfono no esté ya registrado en la tabla '**veterinarios**'; si existe, lanza una excepción.
5. Busca si la especialidad indicada ya existe en la tabla '**especialidades**' por su nombre.
6. Si la especialidad no existe, la crea y captura su '**id_especialidad**' generado. Si ya existe, reutiliza el '**id_especialidad**' existente sin duplicarla.
7. Inserta al veterinario en la tabla '**veterinario**' vinculado al '**id_especialidad**' correspondiente.
8. En caso de cualquier error, ejecuta un ROLLBACK y relanza la excepción con un mensaje descriptivo.



```

CREATE OR REPLACE PROCEDURE sp_registrar_veterinario(

    p_nombre_esp    VARCHAR(150),

    p_descripcion_esp TEXT,

    p_nombre_v      VARCHAR(100),

    p_apellido_v     VARCHAR(100),

    p_telefono_v     VARCHAR(20),

    p_correo_v       VARCHAR(150)

)

LANGUAGE plpgsql

AS $$

DECLARE

    v_id_especialidad BIGINT;

BEGIN

    IF LENGTH(TRIM(p_nombre_esp)) = 0 THEN

        RAISE EXCEPTION 'Error: El nombre de la especialidad no puede estar vacío.';

    END IF;

    IF LENGTH(TRIM(p_nombre_v)) = 0 OR LENGTH(TRIM(p_apellido_v)) = 0 THEN

        RAISE EXCEPTION 'Error: El nombre y apellido del veterinario no pueden estar vacíos.';

    END IF;

    IF EXISTS (SELECT 1 FROM veterinario WHERE correo_v = p_correo_v) THEN

        RAISE EXCEPTION 'Error: El correo % ya está registrado para otro veterinario.', p_correo_v;

    END IF;

    IF EXISTS (SELECT 1 FROM veterinario WHERE telefono_v = p_telefono_v) THEN

        RAISE EXCEPTION 'Error: El teléfono % ya está registrado para otro veterinario.', p_telefono_v;

    END IF;

    SELECT id_especialidad INTO v_id_especialidad

    FROM especialidades

    WHERE nombre_esp = p_nombre_esp;

    IF NOT FOUND THEN

        INSERT INTO especialidades (nombre_esp, descripcion_esp)

        VALUES (p_nombre_esp, p_descripcion_esp)

        RETURNING id_especialidad INTO v_id_especialidad;

    END IF;

    INSERT INTO veterinario (id_especialidad, nombre_v, apellido_v, telefono_v, correo_v)

    VALUES (v_id_especialidad, p_nombre_v, p_apellido_v, p_telefono_v, p_correo_v);

    COMMIT;

EXCEPTION

    WHEN OTHERS THEN

        ROLLBACK;

        RAISE EXCEPTION 'Error al registrar el veterinario: %', SQLERRM;

END;

$$;

```



6.3.2. Agregar medicamento o actualizar stock existente

Objetivo: Registrar un nuevo medicamento en el inventario de la clínica, o bien, actualizar el stock y precio de uno ya existente cuando llega un nuevo lote, evitando duplicados en el catálogo de medicamentos.

Parámetros de entrada:

Parámetro	Tipo	Descripción
p_nombre_m	VARCHAR (150)	Nombre comercial o genérico del medicamento
p_presentacion_m	VARCHAR (100)	Forma farmacéutica (tableta, jarabe, inyectable, etc.)
p_laboratorio_m	VARCHAR (150)	Laboratorio fabricante
p_precio_m	DECIMAL (10,2)	Precio unitario del medicamento
p_cantidad	INT	Cantidad de unidades a registrar o sumar al stock

Lógica:

1. Valida que la cantidad ingresada sea mayor a cero; si no, lanza una excepción controlada.
2. Valida que el precio no sea negativo; si lo es, lanza una excepción controlada.
3. Busca si ya existe un medicamento con el mismo nombre y presentación en la tabla '**medicamentos**'.
4. Si el medicamento **no existe**, lo inserta como nuevo registro con el stock inicial igual a la cantidad recibida.
5. Si el medicamento **ya existe**, suma la cantidad al '**stock_m**' actual y actualiza el '**precio_m**' con el nuevo valor recibido.
6. En caso de cualquier error, lanza una excepción descriptiva.

```

CREATE OR REPLACE PROCEDURE sp_agregar_medicamento(
    p_nombre_m    VARCHAR(150),
    p_presentacion_m VARCHAR(100),
    p_laboratorio_m VARCHAR(150),
    p_precio_m    DECIMAL(10,2),
    p_cantidad    INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_id_medicamento BIGINT;
BEGIN
    IF p_cantidad <= 0 THEN
        RAISE EXCEPTION 'Error: La cantidad debe ser mayor a 0. Recibido: %', p_cantidad;
    END IF;
    IF p_precio_m < 0 THEN
        RAISE EXCEPTION 'Error: El precio no puede ser negativo. Recibido: %', p_precio_m;
    END IF;
    SELECT id_medicamento INTO v_id_medicamento
    FROM medicamentos
    WHERE nombre_m = p_nombre_m
        AND presentacion_m = p_presentacion_m;
    IF NOT FOUND THEN
        INSERT INTO medicamentos (nombre_m, presentacion_m, laboratorio_m, precio_m, stock_m)
        VALUES (p_nombre_m, p_presentacion_m, p_laboratorio_m, p_precio_m, p_cantidad);
    ELSE
        UPDATE medicamentos
        SET stock_m = stock_m + p_cantidad,
            precio_m = p_precio_m
        WHERE id_medicamento = v_id_medicamento;
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Error al agregar el medicamento: %', SQLERRM;
END;
$$;

```



6.4. Disparadores

En esta sección se detallan las funciones de trigger y sus respectivos disparadores creados para automatizar las reglas de consistencia interna y lógicas de negocio.

6.4.1. Control de entidades débiles (Secuenciadores)

Como las tablas detalles_facturas y detalles_tratamientos dependen por completo de sus tablas padres (facturas y tratamiento), se decidió no usar números automáticos tradicionales para sus id's. Estos dos triggers se activan ANTES (BEFORE) de guardar el dato para calcular dinámicamente el número de id (1, 2, 3...).

6.4.1.1. Automatización para DETALLES_FACTURAS

Tipo de trigger: BEFORE INSERT

Lógica: Antes de que se inserte un nuevo registro o servicio en detalles_facturas, el trigger revisa cuál fue la última fila en detalles_facturas relacionadas a esa misma factura y le suma 1. Si no hay ningún registro previo para esa factura, empieza en 1.

```
CREATE OR REPLACE FUNCTION tg_secuencia_detalle_factura()
    RETURNS TRIGGER AS $$
BEGIN
    SELECT COALESCE(MAX(id_detalle_factura), 0) + 1
    INTO NEW.id_detalle_factura
    FROM detalles_facturas
    WHERE id_factura = NEW.id_factura;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_antes_insertar_detalle_factura
    BEFORE INSERT ON detalles_facturas
    FOR EACH ROW
EXECUTE FUNCTION tg_secuencia_detalle_factura();
```



6.4.1.2. Automatización para DETALLES_TRATAMIENTOS

Tipo de trigger: BEFORE INSERT

Lógica: Funciona igual que el anterior. Cuando el veterinario receta varias medicinas dentro de un mismo tratamiento, el trigger se encarga de enumerar las filas (1, 2, 3...) de forma automática para ese tratamiento.

```
CREATE OR REPLACE FUNCTION tg_secuencia_detalle_tratamiento()
    RETURNS TRIGGER AS $$
BEGIN
    SELECT COALESCE(MAX(id_detalle_tratamiento), 0) + 1
    INTO NEW.id_detalle_tratamiento
    FROM detalles_tratamientos
    WHERE id_tratamiento = NEW.id_tratamiento;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_antes_insertar_detalle_tratamiento
    BEFORE INSERT ON detalles_tratamientos
    FOR EACH ROW
    EXECUTE FUNCTION tg_secuencia_detalle_tratamiento();
```

6.4.2. Trigger para sumar el total de la factura automáticamente

Tipo de Trigger: AFTER INSERT.

Propósito: Mantener la sincronización y la integridad de los datos financieros entre la tabla (detalles_facturas) y la tabla padre (facturas), automatizando la suma de subtotales.



Lógica: Realiza una sentencia UPDATE sobre la tabla padre incrementando el valor del atributo total_f mediante la suma de su valor actual con el nuevo subtotal (NEW.subtotal_df).

```
CREATE OR REPLACE FUNCTION tg_actualizar_total_factura()
    RETURNS TRIGGER AS $$
BEGIN
    UPDATE facturas
    SET total_f = total_f + NEW.subtotal_df
    WHERE id_factura = NEW.id_factura;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_acumular_total_factura
    AFTER INSERT ON detalles_facturas
    FOR EACH ROW
EXECUTE FUNCTION tg_actualizar_total_factura();
```

6.4.3. Reducción de stock de medicamentos

Tipo de Trigger: AFTER INSERT.

Propósito: Evitar inconsistencias en la tabla de medicamentos (como saldos negativos de insumos) y automatizar el descuento del inventario ante una venta en caja.

Lógica Técnica:

1. **Evaluación de condicional:** Verifica si el ítem facturado posee un valor no nulo en id_medimento (debido a que un detalle también puede cobrar consultas o procedimientos).



2. **Validación de existencias:** Realiza una lectura del stock_m actual. Si la cantidad solicitada (NEW.cantidad_df) es superior a la disponible, el disparador aborta la operación lanzando una excepción controlada mediante RAISE EXCEPTION.
3. **Rollback seguro:** Al disparar la excepción, la arquitectura de transacciones del motor de base de datos ejecuta un *Rollback* automático, deshaciendo tanto el intento de inserción del detalle como cualquier operación previa amarrada a la transacción, preservando la consistencia.
4. **Modificación:** Si la condición se cumple, descuenta las unidades directamente de la tabla medicamentos.

```
CREATE OR REPLACE FUNCTION tg_controlar_stock_medicamento()
    RETURNS TRIGGER AS $$
DECLARE
    v_stock_actual INT;
BEGIN
    IF NEW.id_medicamento IS NOT NULL THEN
        SELECT stock_m INTO v_stock_actual
        FROM medicamentos
        WHERE id_medicamento = NEW.id_medicamento;

        IF v_stock_actual < NEW.cantidad_df THEN
            RAISE EXCEPTION 'Error de Inventario: No hay suficiente stock para el medicamento solicitado. Disponible:
            %, Solicitado: %',
                v_stock_actual, NEW.cantidad_df;
        END IF;

        UPDATE medicamentos
        SET stock_m = stock_m - NEW.cantidad_df
        WHERE id_medicamento = NEW.id_medicamento;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_actualizar_inventario_farmacia
    AFTER INSERT ON detalles_facturas
    FOR EACH ROW
```



7. Anexos

- **Scripts SQL en PostgreSQL**

<https://github.com/susan-castillo/bd2026-clinica-veterinaria-grupop/tree/main>

